LAB 007

# TERRAFORM CHALLENGE

# TWO-TIER ARCHITECTURE

---

Build It Yourself

VPC • Public & Private Subnets • NAT Gateway • Web Tier • Backend Tier • ALB • Auto Scaling

*Duration: 1 Week | Graded Lab | Individual Work*

---

# Overview

In Lab 006, you followed a step-by-step guide to build a full AWS infrastructure with Terraform. In this lab, you will design and build a different architecture, a two-tier setup, using what you learned. The requirements below describe what your infrastructure must look like.

> ⚠️ **Warning**
> This is an individual graded lab. You must work alone. Any evidence of copied code between students will result in a zero for all involved parties.

# What You're Building

A two-tier web architecture with a public-facing web tier and a private backend tier:

- **Web Tier (Public):** EC2 instances running Nginx or Apache in public subnets, sitting behind an external Application Load Balancer. These are the servers users access from the internet.
- **Backend Tier (Private):** EC2 instances running Nginx or Apache in private subnets. These servers are NOT accessible from the internet. They can only be reached from the web tier.
- **Bastion Host:** A jump box in a public subnet for SSH access to all instances.

The web tier page should display: **"Web Tier — [Your Full Name]"** with the instance ID and availability zone.

The backend tier page should display: **"Backend Tier — [Your Full Name]"** with the instance ID and availability zone.

> 📝 **Note**
> The web and backend tiers both serve simple HTML pages (like Lab 006). The key difference is WHERE they sit in the network — public vs. private subnets — and HOW they are accessed.

# Architecture Requirements

## Networking

| Component | Requirement |
|---|---|
| **VPC** | 1 VPC with a CIDR block of your choice (not 10.0.0.0/16 — pick something different) |
| **Public Subnets** | 2 public subnets in different Availability Zones |
| **Private Subnets** | 2 private subnets in different Availability Zones |
| **Internet Gateway** | 1 IGW attached to the VPC |
| **NAT Gateway** | 1 NAT Gateway in a public subnet with an Elastic IP (so private instances can reach the internet) |
| **Route Tables** | 1 public route table (0.0.0.0/0 → IGW) associated with both public subnets. 1 private route table (0.0.0.0/0 → NAT GW) associated with both private subnets. |

## Security Groups

| Security Group | Rules |
|---|---|
| **Bastion-SG** | Inbound: SSH (22) from 0.0.0.0/0. Outbound: All traffic. |
| **ALB-SG** | Inbound: HTTP (80) from 0.0.0.0/0. Outbound: All traffic. |
| **Web-SG** | Inbound: HTTP (80) from ALB-SG only. SSH (22) from Bastion-SG only. Outbound: All traffic. |
| **Backend-SG** | Inbound: TCP (3000) from Web-SG only. SSH (22) from Bastion-SG only. Outbound: All traffic. |

> 📝 **Note**
> Notice the Backend-SG: it allows port 3000 (not 80) from Web-SG only, because the backend Express API runs on port 3000. The backend is not accessible from the ALB or the internet. This is the security boundary between your tiers.

## Compute

| Component | Requirement |
|---|---|
| **Bastion Host** | 1 x t2.micro in a public subnet. Must have a public IP. |
| **Web Tier** | Auto Scaling Group with min=1, max=4, desired=2. Launch template with user data that installs Nginx, serves the frontend HTML, and configures a reverse proxy to the backend. Instances in public subnets. (See companion document for the application code.) |
| **Backend Tier** | Auto Scaling Group with min=1, max=3, desired=2. Launch template with user data that installs Node.js and runs the Express API on port 3000. Instances in private subnets. (See companion document for the application code.) |

## Load Balancing

| Component | Requirement |
|---|---|
| **External ALB** | Internet-facing ALB in the public subnets, forwarding HTTP traffic to the web tier target group. |
| **Web Target Group** | Health check on / with the web tier ASG instances registered. |

> ⭐ **Bonus**
> Add an Internal ALB in the private subnets that sits in front of the backend tier instances. The web tier would forward requests to the internal ALB instead of directly to backend IPs.
>
> This is how production two-tier architectures actually work. Students who implement this will receive bonus marks.

## Secrets Management

- Store at least 2 parameters in SSM Parameter Store (e.g., a database password as SecureString and a config value as String).
- Create an IAM role and instance profile that allows EC2 instances to read the parameters.
- Attach the instance profile to your launch templates.

### Variables & Outputs

Your Terraform code must use variables (no hardcoded values for region, CIDRs, instance type, or key name) and must output at minimum:

- VPC ID
- External ALB DNS name
- Bastion host public IP
- SSM parameter paths

## Required File Structure

Organize your Terraform code into the following files. You may add more files if needed, but these are the minimum:

```
terraform-challenge/
├── main.tf              # Provider + VPC
├── network.tf           # Subnets, IGW, NAT GW, route tables
├── security.tf          # All security groups
├── compute.tf           # Bastion host, launch templates, ASGs
├── alb.tf               # ALB(s), target groups, listeners
├── secrets.tf           # SSM parameters + IAM role
├── variables.tf         # All input variables
├── outputs.tf           # All outputs
├── terraform.tfvars     # Your variable values (DO NOT push to GitHub)
├── .gitignore           # Must ignore: .terraform/, *.tfstate,
*.tfstate.backup, terraform.tfvars
└── userdata/
    ├── web.sh           # Web tier bootstrap script
    └── backend.sh       # Backend tier bootstrap script
```

> ⚠️ **Warning**
> Your .gitignore must exclude terraform.tfvars, .terraform/, *.tfstate, and *.tfstate.backup. Pushing credentials or state files to GitHub will result in a grade deduction.

# Hints

No code is provided, but here are some hints to guide you:

- **Different VPC CIDR:** Use something like 10.1.0.0/16, 172.16.0.0/16, or 192.168.0.0/16. Plan your subnet CIDRs accordingly (4 subnets = 4 different CIDR blocks).
- **2 AZs everywhere:** Both your public subnets and private subnets should span 2 AZs. Use the aws_availability_zones data source.
- **ASG + ALB link:** Remember the target_group_arns argument in the ASG — this is how instances auto-register with the load balancer.
- **Backend in private subnets:** The backend ASG's vpc_zone_identifier should point to the private subnets, not the public ones.
- **SG-to-SG referencing:** Use security_groups = [aws_security_group.xxx.id] instead of CIDR blocks to restrict access between tiers.
- **User data encoding:** Remember: aws_instance uses file(), but launch templates need base64encode(file()).
- **NAT Gateway placement:** The NAT Gateway goes in a public subnet, but the private route table points to it. Don't mix these up.
- **Terraform docs:** The Terraform AWS provider documentation (registry.terraform.io/providers/hashicorp/aws/latest/docs) is your best friend. Use it.

# Key Differences from Lab 006

This is NOT a copy of Lab 006 with minor tweaks. Here is what has changed:

| Aspect | Lab 006 | Lab 007 | Why |
|---|---|---|---|
| **VPC CIDR** | 10.0.0.0/16 | Your choice (anything except 10.0.0.0/16) | Proves you understand CIDR planning |
| **Private Subnets** | 1 private subnet | 2 private subnets in 2 AZs | Multi-AZ backend for high availability |
| **Architecture** | Single tier (web only) | Two tiers (web + backend) | Real-world separation of concerns |

| | | | |
|---|---|---|---|
| **Backend Tier** | Did not exist | ASG with instances in private subnets | New resource you must design yourself |
| **Security Groups** | 3 SGs | 4 SGs (added Backend-SG) | Tier-to-tier access control |

# Submission Instructions

Submit the following by the deadline:

## Part 1: GitHub Repository

Push all your Terraform files to a GitHub repository named terraform-challenge. Make sure terraform.tfvars and .terraform/ are NOT in the repo (use .gitignore). The repo must contain: main.tf, network.tf, security.tf, compute.tf, alb.tf, secrets.tf, variables.tf, outputs.tf, userdata/web.sh, userdata/backend.sh, and .gitignore.

## Part 2: Screenshots (combined into a single PDF)

Before taking any screenshots, edit your user data scripts to include your full name on the web pages.

**Screenshot 1:** Terminal showing the full output of terraform apply completing successfully, including all output values.

**Screenshot 2:** Browser showing the Two-Tier App page via the external ALB DNS name. Your name must be visible, and the Backend Health Check card should show "healthy" with the backend instance ID.

**Screenshot 3:** Browser refreshed showing a different backend instance ID in the health check or data card (proving the backend ASG has multiple instances responding).

**Screenshot 4:** AWS Console showing the EC2 instances page with your bastion, web tier, and backend tier instances running.

**Screenshot 5:** AWS Console showing the external ALB and its target group with healthy web tier targets.

**Screenshot 6:** AWS Console showing the VPC with all 4 subnets (2 public, 2 private) and their CIDR blocks.

**Screenshot 7:** AWS Console showing the SSM Parameter Store with your parameters created.

**Screenshot 8:** Terminal output of SSHing from the bastion to a backend instance, then running curl http://localhost:3000/api/health to show the backend API responding with JSON.

**Screenshot 9:** Terminal showing terraform destroy completing successfully with the summary of destroyed resources.

**Submit:** The GitHub repository link AND the screenshots PDF through the submission link below.

> ⚠️ **Warning**
> Don't forget to run terraform destroy when you're done to avoid AWS charges.

---

*Good luck. You know more than you think you do.* 🚀